Setting Up Your Computer (Summer 2026)

Learning Objectives

- Install some of the core software we will need in this class.
- Start becoming familiar with Command Line Interfaces (CLI)

Class Goals today:

A. Install Windows Subsystem Linux (WSL) on Windows machines
B. Explore Ubuntu Linux terminal and bash commands
C. Install miniconda package management system
D. Conda Commands
E. Obtaining all Course materials from GitHub
F. Create a virtual (Conda) environment from the CHEM3351.yml file
G. Connecting Conda Environments to Jupyter Lab and Jupyter Notebook

Note: If you have a Mac you do not need to install WSL/Ubuntu because MacOS Terminal uses a Unix-based shell (bash or zsh), which is very similar to Linux.

Note: These setup directions were from resources available through August 4, 2025. As technology is constantly changing, we may need to update these directions as we set up the computers. Please note that Dr. B. set his workstation up in June 2025, and may not have the exact same setup as you.

Note: These directions are based on setup directions generated by Robert Belford at University of Arkansas, Little Rock as part of his Python for Science course. The original can be found here:

https://rebelford.github.io/IOCT-TECH/01_aSetUpComputer%20.html

A. Installing Linux via WSL

****NOTE: You do not Install WSL if you have a Mac****

Before installing WSL, ensure your computer meets these requirements:

Component	Windows Users (for WSL2)
OS Version	Windows 10 (version 1903+) or Windows 11
CPU	64-bit with virtualization support
RAM	At least 4 GB (8+ GB recommended) At least 8 GB RAM for basic usage and exercises in this class 16 GB RAM or more recommended for intensive machine learning activities
Disk Space	At least 10 GB free for tools & software

WSL will automatically install the Ubuntu distribution of Linux by default, but you can choose other distributions if desired. Note that while WSL can run on traditional HDDs, an SSD is recommended for better performance.

https://learn.microsoft.com/en-us/windows/wsl/install

https://learn.microsoft.com/en-us/windows/wsl/setup/environment

Why install WSL when you can use Windows?

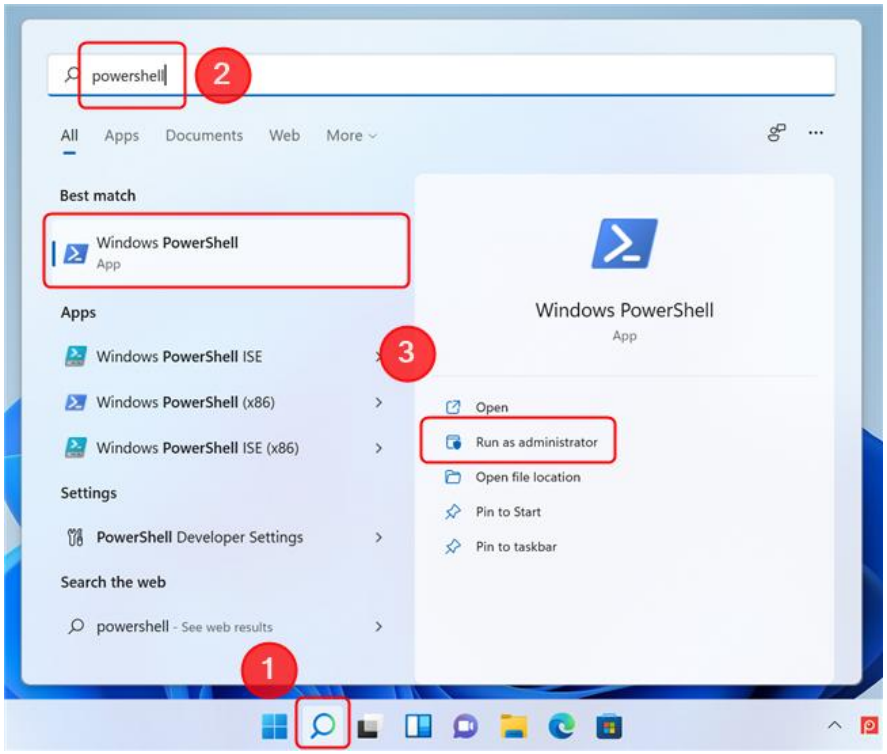
Although python can be installed directly on Windows there are several reasons we are using WSL

- Many Python Libraries are better supported with Linux OS than Windows
- Linux is a standard environment for open-source software production
- WSL allows Linux to integrate with Windows file system (in contrast to a dual boot setup).
- Python scripts often run faster on WSL compared to native Windows setup
- Learning Linux Bash scripts is a useful skill

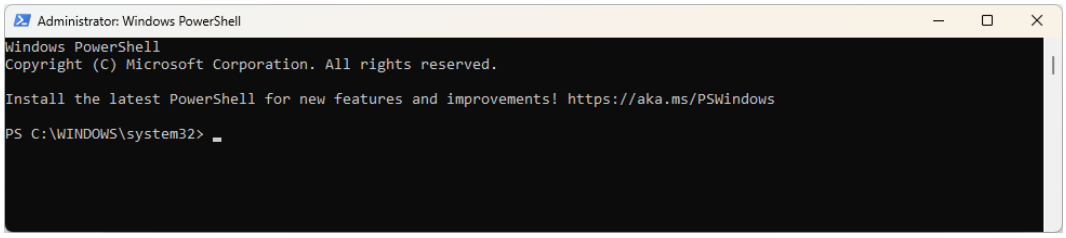
Installation of WSL takes about 3GB of disk space. Make sure you have enough space for this extension of the operating system and course materials.

1. Open Windows PowerShell or Windows Command Prompt in **administrator** mode by right-clicking and selecting "Run as administrator".

To open Windows PowerShell, use Search and type powershell. When you want to run it, click on the Windows PowerShell result. To run as administrator, you must right-click and select "Run as administrator".

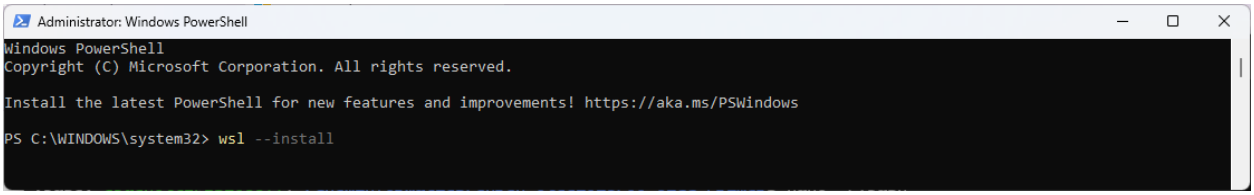


You should have a window that pops up that looks like this:



2. In the PowerShell type:
   `wsl --install`

   Note: When you see `Highlighted Text` these will be commands you need to paste into PowerShell and eventually the Ubuntu operating system.



This command will enable the features necessary to run WSL and install the Ubuntu distribution of Linux.

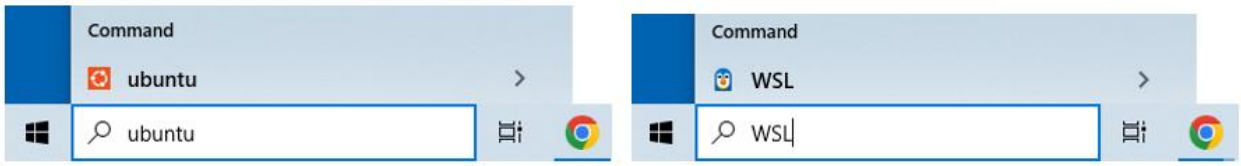
The `--install` command performs the following actions:

- Enables the optional WSL and Virtual Machine Platform components
- Downloads and installs the latest Linux kernel
- Sets WSL 2 as the default
- Downloads and installs the Ubuntu Linux distribution (reboot may be required)

It will take a few minutes to install.

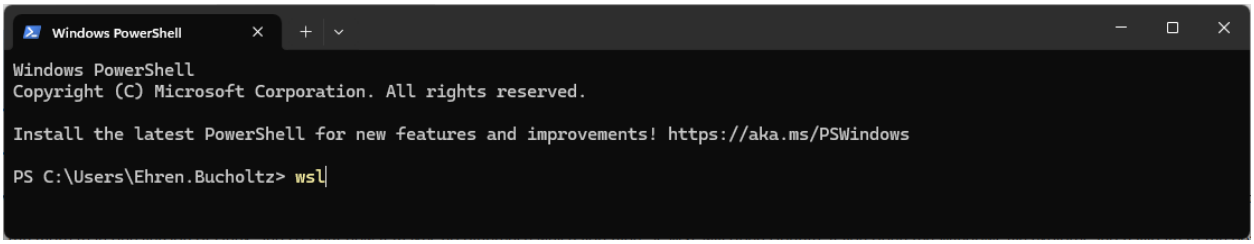
You will need to restart your machine during this installation process. It will put up a screen saying configuring computer.

3. Once installed you can start Ubuntu through the Windows search bar (by typing Ubuntu).



Note, Typing either ubuntu or WSL will start Ubuntu on your windows machine as Ubuntu is the only version of Linux we are running. To avoid issues if another version is installed it is suggested that you always start with Ubuntu, and not WSL

Alternatively, you can start Ubuntu through a Windows PowerShell as yourself (not administrator) and type **wsl.**



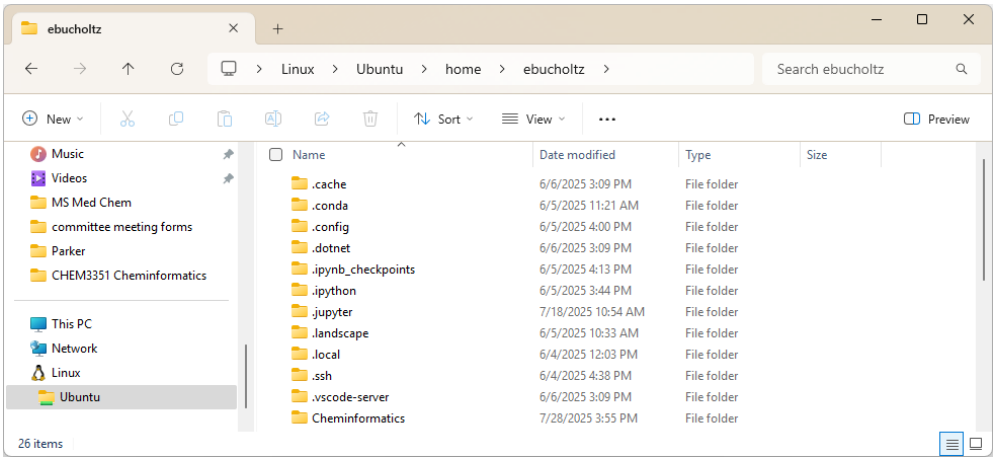
If you get an error like:

```
WslRegisterDistribution failed with error: 0x80370102 Please enable the Virtual Machine Platform Windows feature and ensure virtualization is enabled in the BIOS.
```

Click here if you get this error message.

On some machines, I have received an error message that indicates Windows Subsystem for Linux has no installed distributions. I went back into Powershell as Administrator and retyped wsl --install. This gave a new note saying “Provisioning the new WSL instance ubuntu. This might take a while …” and popped up a new window with information about wsl.

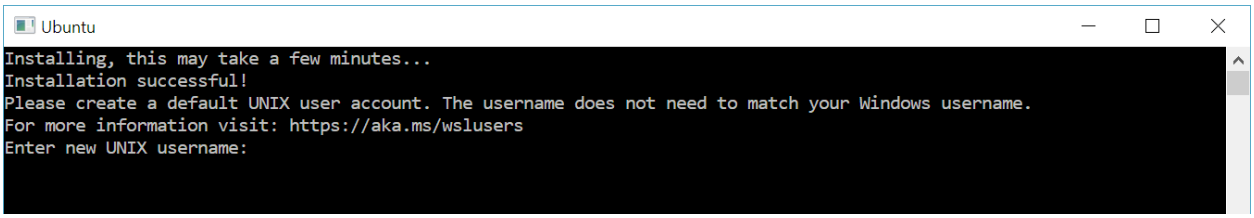
Check the troubleshooting installation article if you run into any issues.



WSL allows you to run more than one version of Linux, and so right now it makes no difference which you type, as WSL will open Ubuntu. If you open the Windows file explorer and navigate to the bottom you can see the directory structure and how WSL is at the very bottom. There will be a Linux Icon at the bottom of your file explorer and your files can be found in the path \\wsl.localhost\Ubuntu\home\username (e.g. ebucholtz). It is in this folder where you will be saving all your course materials.

4. Once you have installed WSL, you will need to create a user account and password for your newly installed Linux distribution.

   In the search



Set up your Linux username and password

Once the process of installing your Linux distribution with WSL is complete, open the distribution (Ubuntu by default) using the Start menu. You will be asked to create a **User Name** and **Password** for your Linux distribution.

- This **User Name** and **Password** is specific to each separate Linux distribution that you install and has no bearing on your Windows username.
- Please note that while entering the **Password**, nothing will appear on screen. This is called *blind typing*. You won't see what you are typing, this is completely normal.
- Once you create a **Username** and **Password**, the account will be your default user for the distribution and automatically sign-in on launch.
- This account will be considered the Linux administrator, with the ability to run `sudo` (Super User Do) administrative commands.
- Each Linux distribution running on WSL has its own Linux user accounts and passwords. You will have to configure a Linux user account every time you add a distribution, reinstall, or reset.

If you need to change or reset your password, or you have forgotten it, click here to learn more.

B. Explore Ubuntu Linux terminal and bash commands

Before we install Miniconda3 we need to update our packages and versions. Note, we could use Anaconda or Anaconda Navigator, but in this class you need to use Miniconda. Miniconda is a minimal installation of Anaconda, which comes with many packages preinstalled and I want you to learn how to install and maintain your packages. Anaconda Navigator is a GUI (Graphical User Interface) for running Anaconda and I want you to gain experience with CLI (Command Line Interfaces), I also find it to be slower to load, and wastes computer resources.

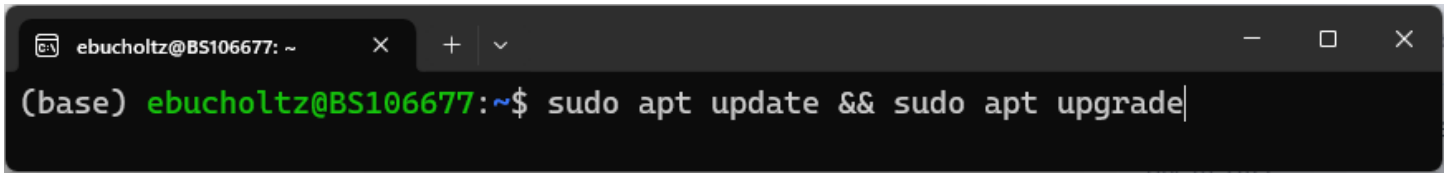
1. Update Packages

In your Ubuntu terminal, type `sudo apt update && sudo apt upgrade`

This command does two things:

- *sudo apt update*: Updates the list of available Linux packages and their versions. You may need your password the first time you run this
- *sudo apt upgrade*: Upgrades the installed packages to their latest versions.

Running this before installing new software ensures you're working with the most up-to-date linux system packages, which can prevent compatibility issues.



Note: my terminal says (base) because I have already installed miniconda.

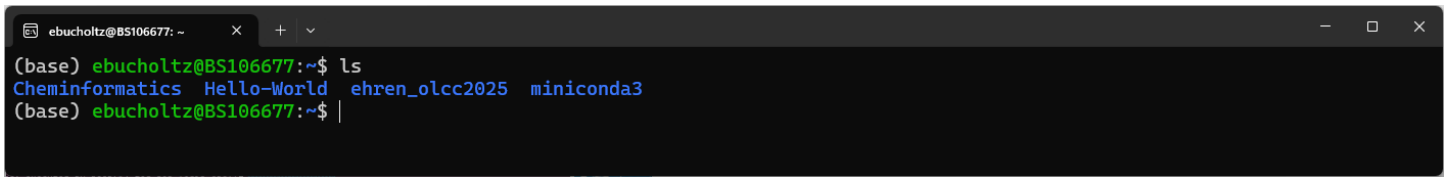
Explanation of the update command:

- *sudo*: Gives you temporary administrative privileges. (Super User Do
- *apt*: The package management tool for Ubuntu.
- *&&*: Runs the second command only if the first one succeeds.

2. Update Bash to always start in your Linux home directory and not your Windows directory

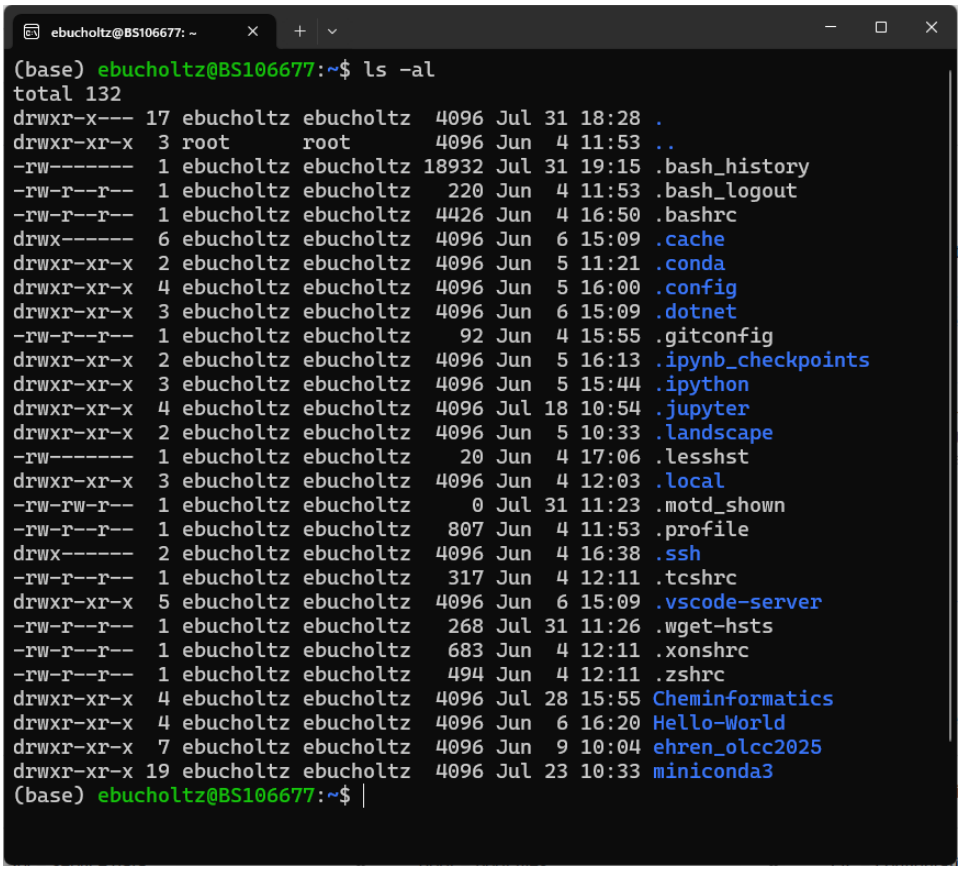
In your linux terminal, we are going to update the configuration script for the Bash (Bourne Again Shell). It is a hidden file called .bashrc that is located in a user's home directory.

To demonstrate it is hidden, first type `ls` in your terminal:



You can see the output here indicates that I have 4 directories (folder) as indicated by the color blue.

When I type `ls -al`, you can see that I have many hidden files.



We are going to use a text editor to edit the .bashrc file

Type `nano ~/.bashrc` and press enter

To edit it, watch me demonstrate in class.

At the bottom of the file add `cd ~`

Save and close the file: Press Ctrl + O, then Enter, and finally Ctrl + X.

Next time you start WSL/Ubuntu it will start in your home directory.

Some Common File Management Bash Commands

You should use this section as a reference, and proceed to section (C) Installing Miniconda.

- pwd Print Working Directory (shows where you are in the file tree)
- ls list files and directories
  - ls -l -a long list including hidden files
  - ls ../ (reaches up one level)
- cd change directories
  - cd ~ navigate to home directory
  - cd / navigate to root directory
  - cd .. go up one level
  - cd - go down one level
- mkdir <directory_name> make a directory in current location
- rmdir <directory_name> remove a directory
  - rmdir -d <directory_name> remove empty directory
  - rmdir -r <directory_name> recursive, removes files and subfolders but asks for confirmation
  - rmdir -rf <directory_name> recursive and forced (does not ask for confirmation)
- rm <filename.extension removes files
- mv <file_name><directory_name> moves file to new directory
- exit closes terminal

NOTE: there are many bash commands associated with installing and managing packages and we will preferentially use Conda commands, like conda install <package-name>, although sometimes we will need to use bash commands like pip install <package-name> which involves the Python Package Index. Usually, you have both options and you should first try the conda option.

For the purposes of this course, I have tried to provide you with a file that will set up your environment and you should not need to do conda or pip installs.

C. Install miniconda package management system

First make sure you are in your user home directory (/home/username).

1. Type: `pwd` print working directory, which should return your user directory (where you are). If you are not in your user home directory (/home/username), type `cd ~` to navigate to it
2. Type: `ls` - list files and subdirectories in current directory
3. Install Miniconda3 in your user directory. When you install miniconda here, a new directory will be created for miniconda. You do not need to make this directory yourself, it will automatically be created .

For Linux users:

Type: `curl -O https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh`

Type: `bash Miniconda3-latest-Linux-x86_64.sh`

For Mac users: (note: I do not have a Mac, and so cannot test the instruction, but they seem logical to me)

Type: `curl -O https://repo.anaconda.com/miniconda/Miniconda3-latest-MacOSX-x86_64.sh`

Type: `bash Miniconda3-latest-MacOSX-x86_64.sh`

- You will need to agree to the license and may have to scroll through it (use the down arrow).
- You will also need to confirm where location miniconda is being installed.
- You will also want to update the shell profile to initialize conda when the command prompt is activated.

Test your installation

1. Type: `exit` to close the terminal
2. Open Ubuntu terminal from the start menu
3. if you see (base) in front of the command prompt, conda is initialized.

If you do not see it, type `conda init` and you should now see (base) in front of the primary prompt.

__conda command not found error:__ If you get this error, refer to article "Conda Command Not Found" by Peter Sawe. You will need to add the path to miniconda3 to your .bashrc file (bash run commands), and this will ensure that Conda and Python will use the versions installed by miniconda.

D. Common Conda Commands

- **Official Conda Cheat Sheet**
- **Official Conda Getting Started Tutorial** (Note: follow this tutorial below for creating your environment for this class).

Updating Conda
When you update conda in the base environment you want to run two commands in your terminal.

1. Type: `conda update -n base -c defaults conda`
2. Type: `conda update --all`

Now lets look at what each of these steps does.
`conda update -n base -c defaults conda`
This command updates conda itself in the base environment:

- `conda update`: This is the basic command to update packages.
- `-n base`: Specifies that the update should occur in the "base" environment, which is the default conda environment.
- `-c defaults`: Instructs conda to use the default channel for the update.
- `conda`: This is the package being updated, which is conda itself.

E. Obtaining all Course materials from GitHub

GitHub (https://github.com/) is a website widely used for sharing and organizing files for coding, data analysis, and other technical projects. In this course, we will use GitHub as the main way to obtain notebooks, data files, and other course materials rather than downloading files individually through Brightspace. This approach will make it easier for you to keep all course materials together in one organized folder on your computer and will also allow us to distribute updates more efficiently if files change during the semester. You will be using GitHub simply as a way to copy course materials to your local machine; you will not be expected to upload work, manage versions, or use GitHub as a submission tool for this course.

I have created a public repository for this course called CHEM3351 (https://github.com/ebucholtz/CHEM3351) that I will use to maintain course files. The way we will download these files is through an application called git. Git is already installed in your ubuntu distribution, but we will check to make sure by typing at the (base) command prompt the following:

`git --version`

It should give you output which resembles the following:

```
(base) ebucholtz@BS106677:~$ git --version
git version 2.43.0
(base) ebucholtz@BS106677:~$
```

To download the files, you will “clone” the repository. In your base environment type:

`git clone https://github.com/ebucholtz/CHEM3351_Summer2026.git`

After you have cloned the repository, type `ls` at the prompt to see that you have a new directory called CHEM3351_Summer2026

```
(base) ebucholtz@BS106677:~$ ls
CHEM3351              CHEM3351_student                Hello-World     development_olcc2025  webscraping
CHEM3351_Summer2026   DataChemistry2025OLCC_Instructor  Untitled.ipynb  miniconda3
(base) ebucholtz@BS106677:~$
```

Change into the CHEM3351_Summer2026 directory and list files by using the following commands:

`cd CHEM3351_Summer2026`

`ls`

It should look like this:

```
(base) ebucholtz@BS106677:~$ cd CHEM3351_Summer2026/
(base) ebucholtz@BS106677:~/CHEM3351_Summer2026$ ls
CHEM3351.yml  README.md  modules
(base) ebucholtz@BS106677:~/CHEM3351_Summer2026$
```

Here is the tricky part. This repository is for course materials only. You must copy/save any notebook that you plan to edit (work on) into a separate `student_work/` folder. To make it easier for all of us when I change files, the original files will always revert back to the original state, or a new version. If you make changes, and you follow the directions to update these modules, you will overwrite your work. Therefore, always save your modules into this new student_work/ folder. Let’s create that folder now by typing

`mkdir  student_work`

```
(base) ebucholtz@BS106677:~/CHEM3351_Summer2026$ mkdir student_work
(base) ebucholtz@BS106677:~/CHEM3351_Summer2026$ ls
CHEM3351.yml  README.md  modules  student_work
(base) ebucholtz@BS106677:~/CHEM3351_Summer2026$
```

Whenever I make an update you will use the command

`git pull`

This will “pull” any changes to the course module files, and will not affect your student_work directory.

Every time you work on a file, you should open it and save it with a new name.

F. Create a virtual (Conda) environment from the OLCC2025.yml file

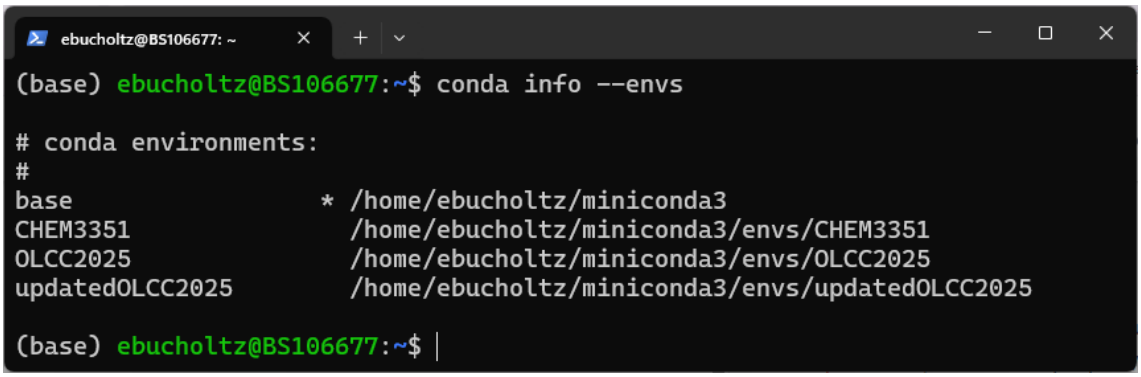
Overview of Virtual Environments and their Pieces

Our goal is to create a virtual environment we can access through a Jupyter Notebook and I will try and explain this at a high level before we get into creating our environment.

1. **The Environment** is like a container to hold your python interpreter and packages.
2. When you create an environment you create a **python interpreter**, and you can specify what version. You subsequently add the packages and their dependencies, and this is the environment you operate in. But you need to be able to access the environment.
3. In the context of Python, particularly with tools like Jupyter Notebooks, a **"kernel"** refers to the computational engine responsible for executing the code you write. It acts as an intermediary between the user interface (like the Jupyter Notebook or JupyterLab) and the Python interpreter. When you **register the kernel** you are giving Jupyter Lab the ability to find the kernel.

When you create a conda environment, conda will automatically create a unique folder for it in the miniconda3/envs folder. To see existing environments, run the following command.

`conda info --envs`



Note: The asterisk (*) in front of the base environment path indicates that it is the active environment. This is also indicated by the (base) in front of ebucholtz@BS106677:~$ in the command prompt. Also note that there are 3 other environments, and are in the envs folder.

Why have other environments?

The base environment houses Conda itself and its core dependencies. Installing project-specific packages directly into base increases the risk of dependency conflicts. Since different projects require different versions of the same package, it could result in potentially breaking the base environment or other projects.

Building an environment

Conda Channels
A Conda channel is a remote or local location where software packages are stored and from which the Conda package manager can retrieve and install them. Think of it as a repository or a directory containing various Conda packages.

Channel Priority
Before creating a channel we want to configure a few things in base environment that have global settings and will apply to all environments, unless an environment over rules them.

There are several channels that Anaconda uses to acquire channels and Conda Forge is considered as one of the best ones to give priority.

The following two lines of code give Conda Forge Priority. You should run these from the base environment.
Type: `conda config --add channels conda-forge`
Type: `conda config --set channel_priority flexible`

Creating an environment from a .yml file

A Conda environment .yml file, conventionally named environment.yml, is a YAML-formatted text file used to define and describe a Conda environment. (YAML originally stood for "Yet Another Markup Language" but was later changed to "YAML Ain't Markup Language") It is a human-readable data serialization format designed to be easily understandable by humans while also being readily parsed by machines. This file serves as a blueprint for creating, updating, or rebuilding a reproducible software environment.

Key components typically found in an environment.yml file:
- **name**: Specifies the name of the Conda environment.
- **channels**: Lists the Conda channels where packages should be searched for (e.g., defaults, conda-forge).
- **dependencies**: Lists the packages and their specific versions required for the environment. This can include packages installed via conda and potentially those installed via pip.

You have been supplied with a file called CHEM3351.yml. It is the blueprint for all the packages we need for the course this semester. This is found in the files that you “pulled” from GitHub.

Confirm you have the

ls	This will give you the list of all files in the folder. It should have the .yml file.
nano CHEM3351.yml or	This will open the file and show you all the library dependencies are for the environment.

After you are done looking at the file, you can control x to close the file. If you accidentally changed anything, if it asks to save changes, choose no as we don't want any changes from the original.

Now that we have the OLCC2025.yml file we need to make one final command to create our local environment.

Type: conda env create -f CHEM3351.yml -n CHEM3351

This will create an environment called CHEM3351based off the CHEM3351.yml file.

This make take a little time as it will download all the packages needed for the course.

After creating your environment, type conda info --envs to list all your conda environments. You should see two environments: base and CHEM3351

Activating the environment

Type: conda activate CHEM3351. This command activates the environment. You should see a new prompt instead of base in front of your username.

Type: conda update --all

- conda update: Again, this is the basic update command.
- --all: This flag tells conda to update all packages in the current environment to their latest compatible versions.
- conda list --revisions gives the history of the current environment

The last thing we need to do is register the environment as a name so that we can easily recognize it as an ipython kernel.

Type: `python -m ipykernel install --user --name=CHEM3351 --display-name "Python (CHEM3351)"`

You should have something that looks like:

```
(CHEM3351) ebucholtz@BS106677:~$ python -m ipykernel install --user --name=CHEM3351 --display-name "Python (CHEM3351)"
Installed kernelspec CHEM3351 in /home/ebucholtz/.local/share/jupyter/kernels/chem3351
(CHEM3351) ebucholtz@BS106677:~$
```

The last thing we will do is just check the version of python we installed.

Type: `python --version`

The latest stable version of Python is 3.13.13, released on April 7, 2026. There is a version 3.14 that was released last fall, but since everything is developed using 3.13, we are using that version.

G. Connecting Conda Environments to Jupyter Lab and Jupyter Notebook

Now that you have a conda environment, we can launch jupyter lab or jupyter notebook to create workflows in that environment as those were installed with the other dependencies. JupyterLab and Jupyter Notebooks run as a local web application accessible through a web browser, typically on localhost. This means the server process is running on your own computer, and you access its interface using a web browser also on your local machine.

Type: `jupyter lab`

This will give you a series of execution prompts in your shell window. You should see something that looks like :

```
    To access the server, open this file in a browser:
        file:///home/ebucholtz/.local/share/jupyter/runtime/jpserver-898-open.html
    Or copy and paste one of these URLs:
        http://localhost:8888/lab?token=3501e9e5fb59e3645402ce0a559abd2b9f2b5a371b887016
        http://127.0.0.1:8888/lab?token=3501e9e5fb59e3645402ce0a559abd2b9f2b5a371b887016
```

The token is a security feature for authenticating user sessions, and serves as a temporary password to prevent unauthorized access to your notebook server.

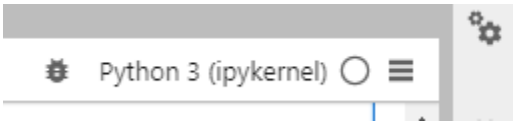
I have found that copying and pasting either the localhost http line or the 127.0.0.1 line in your browser window is most effective at accessing jupyter lab in your browser. You can also ctrl+click on either line and it will launch in your default browser.

You will do this each time you start working on a jupyter notebook for class.

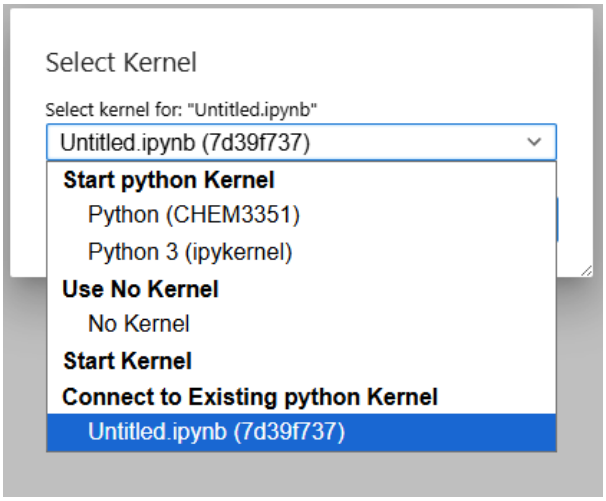
To test that the system is working, in your browser, click File→New→Notebook

Now that you have Jupyter lab installed and have created your first environment, you can go to that environment and open the lab. But you need to connect the kernel to the one you registered in your

environment, which is done by clicking on the box in the upper right corner of your notebook (Python3(ipkernel) in image below).



If you click it a drop down box will appear and you can choose the kernel you wish to use in this notebook. (I believe that Python 3 (ipykernel) is the same as CHEM3351 here, but I am not absolutely sure.)



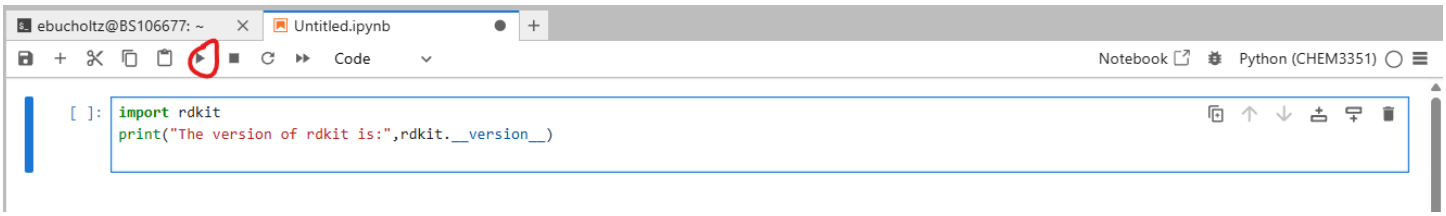
To test that this kernel is working, we will do a test to make sure we can import a dependent library and see what version it is.

In your untitled notebook, copy and paste the following code into the first code cell:

```
import rdkit
print("The version of rdkit is:",rdkit.__version__)
```

You will see that the first code cell has an empty bracket.

Click the play button to run the code:



You should have something returned like:

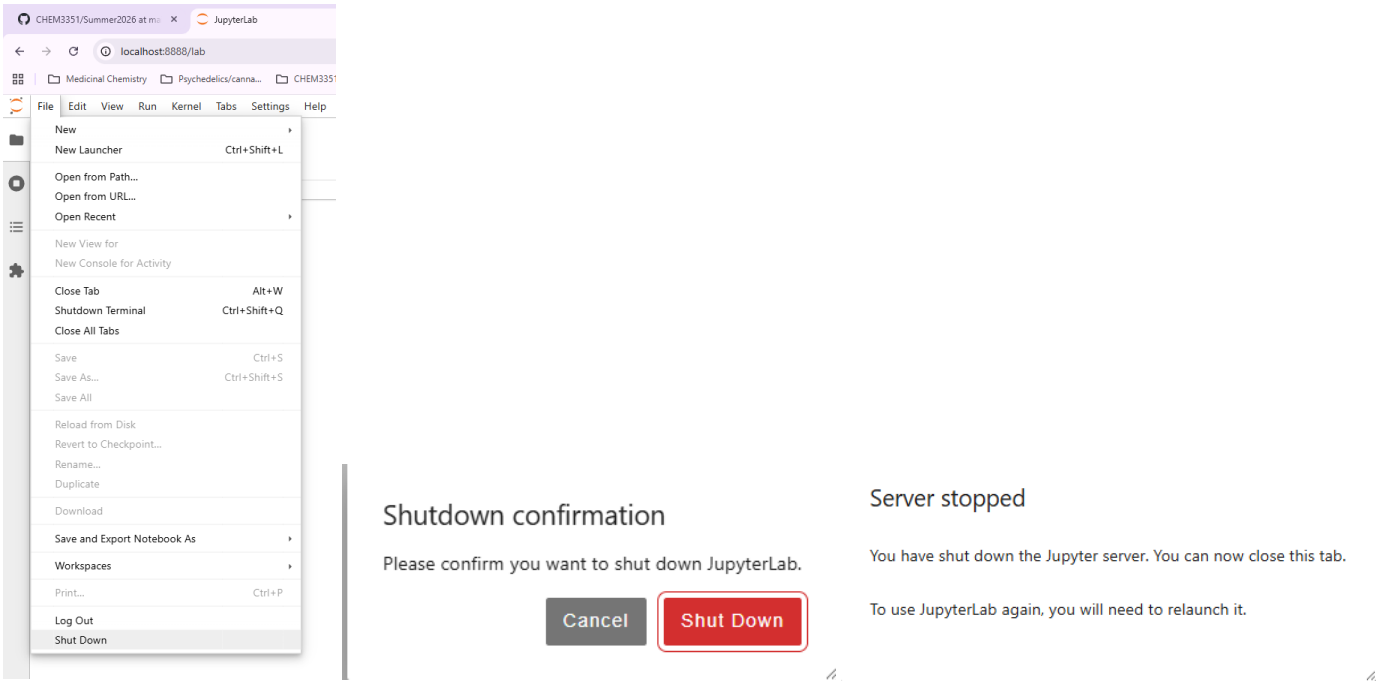
```
[1]: import rdkit
     print("The version of rdkit is:",rdkit.__version__)

     The version of rdkit is: 2025.03.6
```

Notice that the bracket now has a 1 inside indicating you have run the code and that was the first cell run.

If everything is good, you should have output. If not, contact Dr. B.

Now that we know jupyter lab is running properly, have a viable conda environment, we can shut down the conda shell for now. To do so in your Jupyter notebook this this by going to file in Notebook menu, choose shut down and confirm:



Go back to your shell, and you will see that the prompt has returned and the server has shut down.

```
[I 2026-04-17 10:58:09.614 ServerApp] Terminal 1 closed
[I 2026-04-17 10:58:09.614 ServerApp] EOF on FD 17; stopping reading
[I 2026-04-17 10:58:09.715 ServerApp] Terminal 2 closed
[I 2026-04-17 10:58:09.800 ServerApp] Shutting down on /api/shutdown request.
[I 2026-04-17 10:58:09.806 ServerApp] Shutting down 5 extensions
(CHEM3351) ebucholtz@BS106677:~/CHEM3351/Summer2026$
```

Conclusion:

At this end of this setup, you should have installed WSL, set up a conda environment for the class and are ready to use this system for local cheminformatics workflows on your computer.